



36. Python: Requests API

Connect Python to the Web — One Request at a Time

Previous Section:

 [35. Python: Graphviz](#)

Contents:

[1. The Requests Library](#)

[Common HTTP Methods](#)

[Frequently Used Requests Functions](#)

[2. Some Common Operations on Requests](#)

[Make a GET Request](#)

[Inspect the Response](#)

[Add Query String Parameters](#)

[Customize Request Headers](#)

[Use Other HTTP Methods](#)

[Send Request Data](#)

[Use Authentication](#)

[Summary](#)

[References](#)

Modern applications rarely work in isolation. Weather apps retrieve forecasts from online services. Currency converters obtain live exchange rates. News applications fetch articles from remote servers. Social media platforms communicate with countless web services behind the scenes.

To make all of this possible, we need a way for Python to communicate with external websites and web APIs.

This is where the **Requests** library comes in.

Requests is one of the most popular Python libraries for making HTTP requests. It provides a simple and human-friendly interface for interacting with web servers, REST APIs, cloud services, and online applications.

In this section, we will learn how to use Requests to send data, receive responses, authenticate users, and communicate securely with web services.

1. The Requests Library

The Requests library allows us to send HTTP requests to web servers and receive responses.

Before using it, we typically install it with:

```
pip install requests
```

And import it into our Python program:

```
import requests
```

Why do many developers prefer Requests?

- Simple and readable syntax
- Supports all common HTTP methods
- Automatic handling of query parameters
- Easy authentication support
- Built-in HTTPS support
- Widely used in API development

Common HTTP Methods

Method	Purpose
GET	Retrieve information from a server
POST	Send new data to a server
PUT	Update existing data
PATCH	Partially update data
DELETE	Remove data
HEAD	Retrieve headers only
OPTIONS	Discover supported methods

Frequently Used Requests Functions

Function	Description
<code>requests.get()</code>	Send a GET request
<code>requests.post()</code>	Send a POST request
<code>requests.put()</code>	Send a PUT request
<code>requests.delete()</code>	Send a DELETE request
<code>response.status_code</code>	Get HTTP status code
<code>response.text</code>	Retrieve response as text
<code>response.json()</code>	Convert JSON response to Python objects
<code>response.headers</code>	Access response headers
<code>response.url</code>	View the final URL

2. Some Common Operations on Requests

Make a GET Request

The GET method is used when we want to retrieve information from a server.

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1"
)

print(response.text)
```

```
{
  "userId": 1,
  "id": 1,
  "title": "...",
  "body": "..."
}
```

The server sends data back, and Requests stores it inside the `response` object.

Inspect the Response

After receiving a response, we often need to inspect its contents.

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1"
)

print("Status Code:", response.status_code)
print("URL:", response.url)
print("Headers:", response.headers)
```

Status Code: 200

URL: https://jsonplaceholder.typicode.com/posts/1

A status code of **200** typically indicates success.

Common status codes:

Code	Meaning
200	Success

Code	Meaning
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Add Query String Parameters

Many APIs accept parameters through the URL. Instead of manually constructing URLs, we can use the `params` argument.

```
import requests

parameters = {
    "userId": 1
}

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts",
    params=parameters
)

print(response.url)
print(response.json())
```

Generated URL: `https://jsonplaceholder.typicode.com/posts?userId=1`

Requests automatically formats the query string for us.

Customize Request Headers

Headers provide additional information about the request.

```
import requests

headers = {
    "User-Agent": "IrisApp/1.0"
}

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    headers=headers
)

print(response.status_code)
```

Common header examples: User-Agent; Authorization; Content-Type; Accept

Headers are frequently used when working with APIs that require authentication or special formatting.

Use Other HTTP Methods

Requests supports all major HTTP methods.

- POST

```
import requests

data = {
    "title": "My Post",
    "body": "Hello World",
    "userId": 1
}

response = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json=data
)
```

```
print(response.json())
```

- PUT

```
import requests

updated_data = {
    "id": 1,
    "title": "Updated Title",
    "body": "Updated Content",
    "userId": 1
}

response = requests.put(
    "https://jsonplaceholder.typicode.com/posts/1",
    json=updated_data
)

print(response.json())
```

- DELETE

```
import requests

response = requests.delete(
    "https://jsonplaceholder.typicode.com/posts/1"
)

print(response.status_code)
```

Send Request Data

When sending information to a server, we can include data in the request body.

- Form Data

```
import requests

data = {
    "username": "iris",
    "password": "secret"
}

response = requests.post(
    "https://httpbin.org/post",
    data=data
)

print(response.json())
```

- JSON Data

```
import requests

payload = {
    "name": "Iris",
    "role": "Journalist"
}

response = requests.post(
    "https://httpbin.org/post",
    json=payload
)

print(response.json())
```

Using `json=` automatically converts the dictionary into JSON format.

- Inspect the Prepared Request

Sometimes we want to see the exact request that will be sent.

```
import requests

request = requests.Request(
    "GET",
    "https://example.com",
    params={"page": 1}
)

prepared = request.prepare()

print(prepared.method)
print(prepared.url)
```

```
GET
https://example.com/?page=1
```

This is useful for debugging API requests.

Use Authentication

Some APIs require credentials before allowing access.

- Basic Authentication

```
import requests

response = requests.get(
    "https://httpbin.org/basic-auth/user/pass",
    auth=("user", "pass")
)

print(response.status_code)
```

Requests automatically generates the appropriate authentication header.

- Communicate Securely With Servers

Most modern APIs use HTTPS.

```
import requests

response = requests.get(
    "https://api.github.com"
)

print(response.status_code)
```

HTTPS encrypts communication between our application and the server.

Requests verifies SSL certificates automatically, helping protect us from man-in-the-middle attacks. In rare testing environments, SSL verification can be disabled:

```
response = requests.get(
    "https://example.com",
    verify=False
)
```

However, this should generally be avoided in production systems because it reduces security.

Summary

Requests is one of the most useful libraries in Python for communicating with web servers and external APIs.

In this section, we learned how to:

- Send GET requests
- Inspect server responses
- Work with query parameters
- Customize request headers
- Use POST, PUT, and DELETE methods
- Send form and JSON data
- Inspect prepared requests

- Authenticate with APIs
- Communicate securely using HTTPS

With Requests, we can easily connect Python applications to weather services, financial systems, social media platforms, cloud applications, and countless other web-based services. It serves as the foundation for many modern applications that rely on data from external sources.

References

<https://realpython.com/python-requests/>

<https://www.geeksforgeeks.org/python/python-requests-tutorial/>

<https://www.datacamp.com/tutorial/making-http-requests-in-python>

Next Section:

[</> 37. Python: Working with SQL](#)